TEAM AND PROJECT

# Fax Console

A house PBX's legacy fax tools, modernized with IBM Bob while the phone line kept working.

**Track** Modernize What Matters | **Team** Jeffrey Hein, solo | **Built with** IBM Bob

| | |
|---|---|
| Repository | github.com/jphein/fax-console |
| Demo | https://jphein.github.io/fax-console/ (a static replay) |
| Video | https://youtu.be/SnsuG6-jPNQ |

TOOL USAGE & DEPTH OF ADOPTION · 30%

*The rubric asks: "Did the team actually build with IBM Bob as a core part of their workflow, not just bolt it on at the end?"*

# Bob did the modernization, and every run is on record

1

## Analyze

Module map, every shell-out, fragile patterns, a migration order.

`docs/analysis.md`

2

## Package

The CLI as a typed package: pure logic split from I/O.

`faxcli/`

3

## Test

Unit, golden and characterization tests on scrubbed fixtures.

`tests/`

4

## Extract

The Fax panel out of the monolith, as a typed API.

`faxconsole/`

5

## Isolate

One Asterisk adapter, with replay mode on fixtures.

`adapter + replay`

Bob factor

**65.4%**

of the new code and test lines came from Bob's runs

Bob-assisted commits

**11**

one per run that changed files, marked `Assisted-by: IBM Bob`

Bob runs on record

**14**

runs 0 to 13, each with its prompt and transcript

Rejected in review

Run 12's replay mode reported fax calls it never placed. Review reverted it.

**How Bob runs.** Bob Shell, headless, inside an OS sandbox. Claude (Anthropic) drives it and checks every claim against the code; Jeffrey approves at two checkpoints.

*The rubric asks: "Did the team actually build with IBM Bob as a core part of their workflow, not just bolt it on at the end?"*

# Bob from day one: every run, by day




Sep 28, hacking begins

Oct 18, deadline

| DAY | RUNS | WHAT BOB DID | BOBCOINS |
|---|---|---|---|
| **Sep 28** | runs 0 to 6 | Sandbox smoke test · Modernization analysis · Isolation check · Sandbox check · The faxcli package · The review fixes · The faxconsole core | 22.44 |
| **Sep 29** | runs 7 to 13 | Run 6's review and the VoIP.ms poller · Finish the VoIP.ms port · The page · The inbound dialplan · Review fixes · Faxcli fixes · Oracle fixes | 40.68 |
| **So far** | **14 runs** | **on 2 of the 21 days** | **63.12** |

Rebuilt from the ledger in `docs/bob-usage.md` by `docs/deck/timeline.py`. Every run's prompt and transcript are in `docs/bob-runs/`, with a guard log for each run that started. In run 5 the sandbox refused Bob's `git stash`.

INNOVATION & USE CASE RELEVANCE · 30%

*The rubric asks: "Is the use case a credible, realistic application of IBM Bob (vs. a generic project that happens to touch it)?"*

# A realistic job for Bob: legacy code that runs a real phone line

## The legacy system

- `telephony-console.py`: **11,858** lines in one file, with hand-rolled routing
- It shells out for every read: 11 call sites (analysis §4)
- `fax/cli.py`: **288** lines around Asterisk `SendFax`, no tests
- In production on the house PBX: status, the fax log and sending must keep working

### Bob found a real flaw

The legacy inbound-fax design passed the caller's number to a shell. Bob caught it unaided in run 10, and review narrowed the filter to digits and `+`.

### Safe by construction

Bob works in an OS sandbox that cannot reach the LAN or the PBX. In run 5 it tried `git stash`; the read-only `.git` refused it.

### No private data reached an AI tool

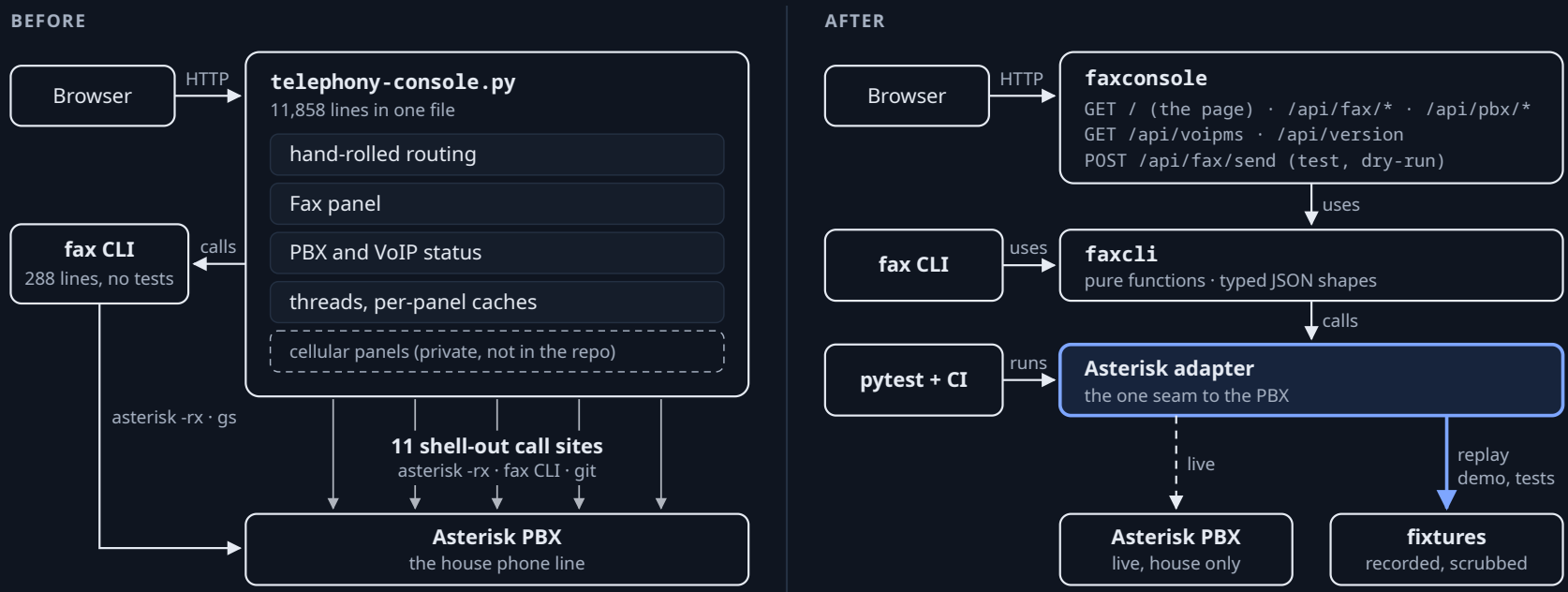
Every prompt passes the scrub gate first. Fixtures use 555-01xx numbers and example.com hosts.

> The pattern carries over to any single-file admin tool: put one seam in front of the live system, record fixtures behind it, and the tool becomes testable and safe to demo.

**FUNCTIONALITY & TECHNICAL EXECUTION · 30%**

*The rubric asks: "Does the project work as demoed? Is it technically sound given the 3-week timeframe?"*

# Before and after, and the proof it works




Before: each reader shells out on its own (11 call sites, analysis §4), and nothing runs without the live line. After: one adapter is the only way in, and replay mode points it at recorded fixtures.

| Tests | Deliberate fixes to legacy | CI on every push | Demo |
|---|---|---|---|
| **1066** | **8** | **passing** | **GitHub Pages** |
| on scrubbed fixtures | each tested, in `docs/changes.md` | Python 3.10, 3.12 and 3.14 | a static replay, read-only |

PRESENTATION & CLARITY · 10%

*The rubric asks: "Can the team clearly explain what they built and how Bob was used in a few minutes?"*

# In one minute, and how to check it yourself

## What we built

A house PBX's legacy fax tools, modernized without taking the phone line down: a typed, tested `faxcli` package, and the Fax panel as a small service (`faxconsole`): a typed API and a page, behind one Asterisk adapter, with a replay mode for the demo.

## How Bob was used

Bob wrote the analysis, both packages, the page and their tests, and fixed most of what review found (Bob factor 65.4%), headless, inside an OS sandbox, with every run recorded.

## Improvements made in the window

Commit #1 is the pre-hackathon baseline, scrubbed and otherwise unchanged, rebuilt once on day one. Everything after it is new. Bob wrote most of the code and tests. Claude wrote the scrub gate, CI, the sandbox and the ledger, plus a few small review fixes, each in its own commit with a test.

```
# View it
Demo (a static replay):
  https://jphein.github.io/fax-console/
Demo video (under 3 minutes):        https://youtu.be/SnsuG6-
jPNQ
The Bob ledger:                      docs/bob-usage.md
Each Bob run, prompt to guard log:   docs/bob-runs/
Deliberate changes from legacy:      docs/changes.md

# Run it (no PBX needed)
$ git clone https://github.com/jphein/fax-console
$ cd fax-console
$ python3 -m faxconsole --replay tests/fixtures
  then open http://127.0.0.1:8093
$ python3 -m pytest -q

# See which commits Bob helped write
$ git log --oneline --grep 'Assisted-by: IBM Bob'
```

FUTURE ROADMAP · KEY LEARNINGS AND NEXT STEPS

# What we learned, and what's next

## LEARNED

- **Scrub before the first prompt.** The baseline was cleaned of numbers, names and hosts before any AI tool saw it, and a pre-commit check keeps it that way.
- **Review Bob against the code.** The analysis's most important finding, a PBX that cannot be read reported as `ok: true`, came from review, not from Bob.
- **Check that a test can fail.** Review found Bob tests that stayed green on broken code in runs 5, 7, 8, 9 and 10. Review now breaks the code on purpose and checks that the tests go red.

## ROOM TO IMPROVE

- **Size Bob's runs to the cap.** Runs 11 to 13 all hit their Bobcoin cap, and twice the reviewer had to finish the work. Smaller, single-task runs would fit.
- **The inbound path is config only.** Run 10's dialplan and hook are generated and tested, but have never run on a live line.

## NEXT

- Deploy the inbound-fax path on a test line that cannot disturb the house number.
- Bring the rest of the console behind the same seam.

**NEXT: LTE ON CBRS**

A Magma core on CBRS Band 48 with SMS and voice fallback through the same PBX. Idea stage, not built.